

CO2 - AT2 - Part I

Industry Problem Task

Dynamic Programming & Monte Carlo Methods

```
summary_table = pd.DataFrame({
    "Mean": df_scaled.mean(),
    "Median": df_scaled.median(),
    "Std Dev": df_scaled.std(),
    "Variance": df_scaled.var(),
    "Minimum": df_scaled.min(),
    "Maximum": df_scaled.max()
})

print(summary_table)
```

...		Mean	Median	Std Dev	Variance	Minimum	Maximum
	Reliability	0.491092	0.480160	0.298295	0.088980	0.000000	1.000000
	Consistency	0.500893	0.499089	0.253928	0.064480	0.000000	1.000000
	Scalability	0.533409	0.565576	0.294312	0.086620	0.000000	1.000000
	Flexibility	0.535411	0.532038	0.284361	0.080861	0.000000	1.000000
	Adaptability	0.542468	0.536177	0.269944	0.072870	0.000000	1.000000
	Redundancy	0.532913	0.529674	0.289774	0.083969	0.000000	1.000000
	Reward	0.522697	0.521126	0.267555	0.071586	0.081165	0.891933

```
sample10 = df_scaled.head(10)
```

```
print(sample10)
```

	Reliability	Consistency	Scalability	Flexibility	Adaptability	\
0	0.518677	0.352870	0.625475	0.432846	0.492085	
1	0.723960	0.502028	0.365046	0.296364	0.573201	
2	0.645891	0.638266	0.541698	0.547615	0.503339	
3	0.334205	0.574902	0.469451	0.776370	0.709863	
4	0.589064	0.531050	0.436318	0.535380	0.561801	
5	0.605325	0.497354	0.656394	0.383432	0.596293	
6	0.521272	0.613866	0.625901	0.467728	0.695607	
7	0.732656	0.500823	0.566633	0.400599	0.438389	
8	0.489198	0.492340	0.572676	0.682489	0.672955	
9	0.409711	0.568473	0.645818	0.539951	0.425846	

	Redundancy	Reward
0	0.593822	0.502629
1	0.459816	0.486736
2	0.641922	0.586455
3	0.432243	0.549506
4	0.521438	0.529175
5	0.329640	0.511406
6	0.563334	0.581285
7	0.520765	0.526644
8	0.564662	0.579053
9	0.548830	0.523105

```
▶ plt.figure(figsize=(12,6))

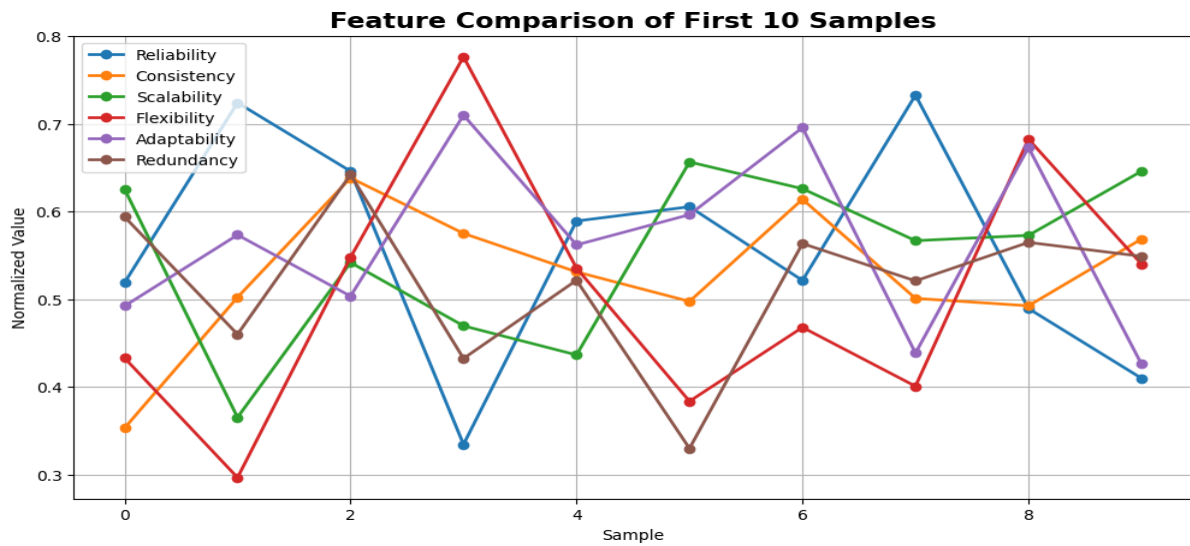
for col in df_scaled.columns[:-1]: # Exclude Reward
    plt.plot(sample10.index,
             sample10[col],
             marker='o',
             linewidth=2,
             label=col)

plt.title("Feature Comparison of First 10 Samples",
          fontsize=16,
          fontweight='bold')

plt.xlabel("Sample")
plt.ylabel("Normalized Value")

plt.legend()
plt.grid(True)

plt.show()
```



```
dp_iteration = pd.DataFrame({
    "State": np.arange(10),
    "Reward": rewards[:10],
    "DP Value": V[:10]
})

print(dp_iteration)
```

	State	Reward	DP Value
0	0	0.502629	5.214926
1	1	0.486736	5.235885
2	2	0.586455	5.276833
3	3	0.549506	5.211530
4	4	0.529175	5.180028
5	5	0.511406	5.167614
6	6	0.581285	5.173564
7	7	0.526644	5.102532
8	8	0.579053	5.084320
9	9	0.523105	5.005852

```

plt.figure(figsize=(10,5))

plt.plot(dp_iteration["State"],
         dp_iteration["DP Value"],
         marker='o',
         color='royalblue',
         linewidth=3)

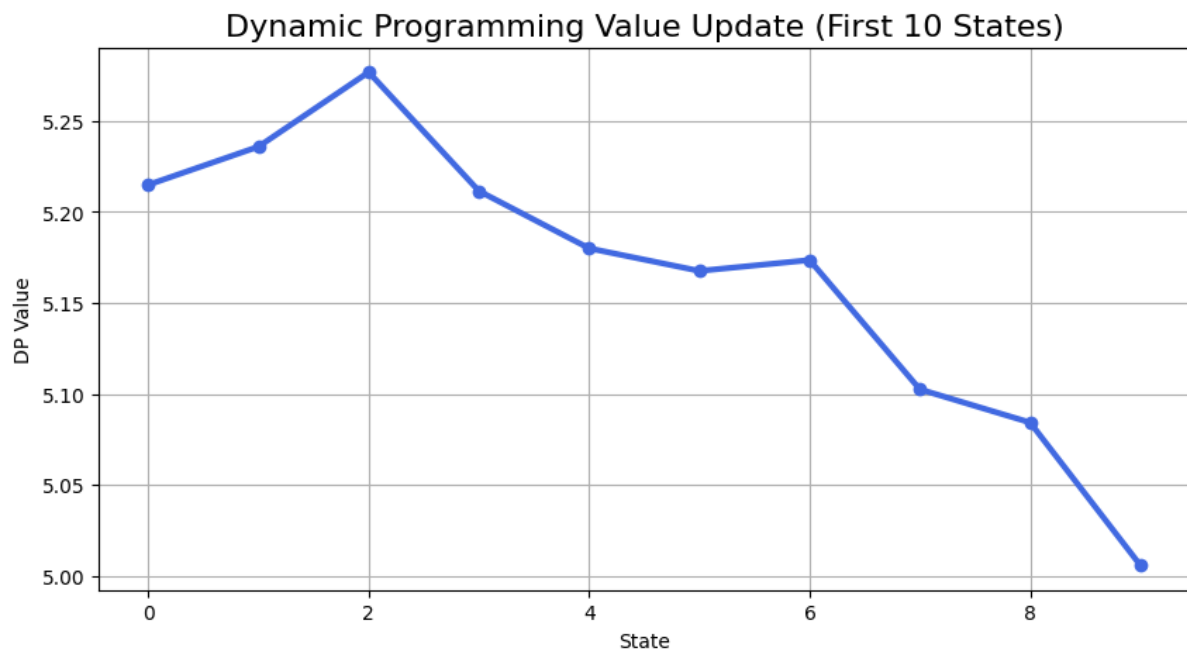
plt.title("Dynamic Programming Value Update (First 10 States)",
         fontsize=16)

plt.xlabel("State")
plt.ylabel("DP Value")

plt.grid(True)

plt.show()

```



```

mc_iteration = pd.DataFrame({
    "State": np.arange(10),
    "Reward": rewards[:10],
    "Monte Carlo Value": mc_values[:10]
})

print(mc_iteration)

```

	State	Reward	Monte Carlo Value
0	0	0.502629	5.214951
1	1	0.486736	5.235913
2	2	0.586455	5.276863
3	3	0.549506	5.211565
4	4	0.529175	5.180066
5	5	0.511406	5.167656
6	6	0.581285	5.173610
7	7	0.526644	5.102584
8	8	0.579053	5.084378
9	9	0.523105	5.005916

```
plt.figure(figsize=(10,5))

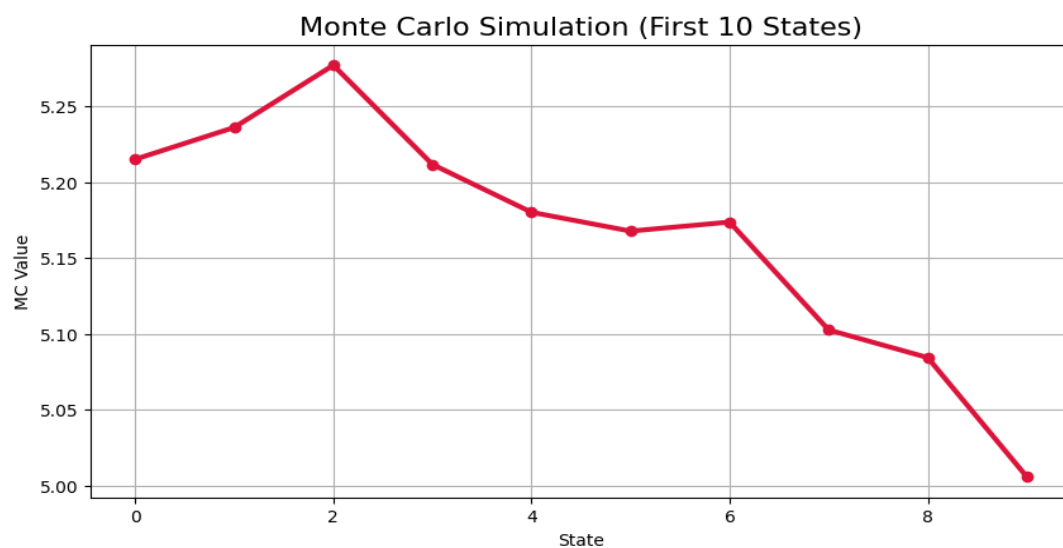
plt.plot(mc_iteration["State"],
         mc_iteration["Monte Carlo Value"],
         marker='o',
         color='crimson',
         linewidth=3)

plt.title("Monte Carlo Simulation (First 10 States)",
         fontsize=16)

plt.xlabel("State")
plt.ylabel("MC Value")

plt.grid(True)

plt.show()
```



```

evaluation = pd.DataFrame({
    "Metric":[
        "Mean",
        "Median",
        "Std Dev",
        "Variance",
        "Execution Time"
    ],
    "Dynamic Programming":[
        np.mean(V),
        np.median(V),
        np.std(V),
        np.var(V),
        dp_time
    ],
    "Monte Carlo":[
        np.mean(mc_values),
        np.median(mc_values),
        np.std(mc_values),
        np.var(mc_values),
        mc_time
    ]
})

print(evaluation)

```

	Metric	Dynamic Programming	Monte Carlo
0	Mean	4.681740	4.757629
1	Median	4.660493	4.752039
2	Std Dev	1.877644	1.869846
3	Variance	3.525546	3.496325
4	Execution Time	0.000218	3.307059

```

from scipy.stats import ttest_ind

t,p=ttest_ind(V,mc_values)

print("T Statistic :",round(t,4))
print("P Value :",round(p,4))

if p<0.05:
    print("Significant Difference")
else:
    print("No Significant Difference")

```

T Statistic : -0.2849
P Value : 0.776
No Significant Difference

```
import matplotlib.pyplot as plt
import numpy as np

metrics = ['Mean', 'Std Dev', 'Execution Time']

dp_metrics = [
    np.mean(V),
    np.std(V),
    dp_time
]

mc_metrics = [
    np.mean(mc_values),
    np.std(mc_values),
    mc_time
]

x = np.arange(len(metrics))
width = 0.35

plt.figure(figsize=(10,6))

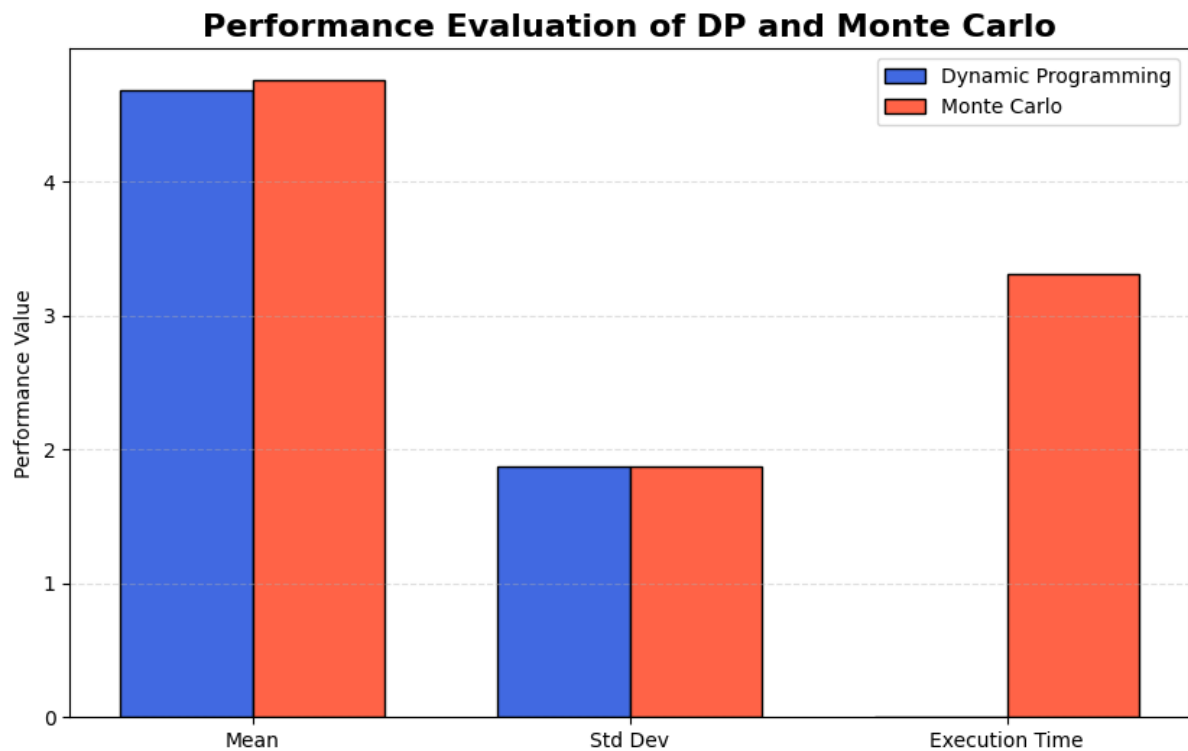
plt.bar(x - width/2, dp_metrics, width,
        color='royalblue',
        edgecolor='black',
        label='Dynamic Programming')
```

```
plt.bar(x + width/2, mc_metrics, width,  
       color='tomato',  
       edgecolor='black',  
       label='Monte Carlo')
```

```
plt.xticks(x, metrics)  
plt.ylabel("Performance Value")  
plt.title("Performance Evaluation of DP and Monte Carlo",  
         fontsize=16,  
         fontweight='bold')
```

```
plt.legend()  
plt.grid(axis='y', linestyle='--', alpha=0.4)
```

```
plt.show()
```




```
dp_score = np.mean(V) / dp_time
mc_score = np.mean(mc_values) / mc_time
```

```
plt.figure(figsize=(7,5))
```

```
bars = plt.bar(
    ['Dynamic Programming', 'Monte Carlo'],
    [dp_score, mc_score],
    color=['royalblue', 'crimson'],
    edgecolor='black'
)
```

```
for bar in bars:
    plt.text(
        bar.get_x() + bar.get_width()/2,
        bar.get_height(),
        f'{bar.get_height():.2f}',
        ha='center',
        fontsize=11
    )
```

```
plt.title("Performance Score Comparison",
        fontsize=16,
        fontweight='bold')
```

```
plt.ylabel("Performance Score")
```

```
plt.grid(axis='y', linestyle='--', alpha=0.5)
```

```
plt.show()
```

Performance Score Comparison

